SIMATS ENGINEERING

ASSESSMENT TOOL 4 - HIGH

(CODING CHALLENGE QUESTIONS)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A

ANALYTICAL PROBLEM SOLVING QUESTIONS

Code of Conduct:

I Yugitha B (192424102) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. **Classify daily temperatures**

```
temperature <- c(18, 25, 35, 20, 30)
for (temp in temperature) {
  if (temp < 20) {
    print("Low")
  } else if (temp <= 30) {
    print("Moderate")
  } else {
    print("High")
  }
}
```

```
+   }
+ }
[1] "Low"
[1] "Moderate"
[1] "High"
[1] "Moderate"
[1] "Moderate"
>
```

2. **Scholarship eligibility**

```
marks <- c(90, 80, 88)
attendance <- c(85, 90, 75)
for (i in 1:3) {
  if (marks[i] >= 85 && attendance[i] >= 80) {
    print(paste("Student", i, "- Eligible"))
  } else {
    print(paste("Student", i, "- Not Eligible"))
  }
}
```

```
+ }
[1] "Student 1 - Eligible"
[1] "Student 2 - Not Eligible"
[1] "Student 3 - Not Eligible"
>
```

3. **Sum of even numbers from 1 to n**

```
n <- as.integer(readline("Enter n: "))
if (is.na(n) || n < 1) {
  print("Please enter a positive integer")
} else {
  total <- 0

  for (i in 1:n) {
    if (i %% 2 == 0) {
      total <- total + i
    }
  }

  print(total)
}
```

```
+ }
[1] "Please enter a positive integer"
> 10
[1] 10
```

4. Reverse the digits of a positive integer

```
n <- 1234

reverse <- 0

while (n > 0) {

  digit <- n %% 10

  reverse <- reverse * 10 + digit

  n <- n %/% 10

}

print(reverse)
```

```
> n <- 1234
> reverse <- 0
>
> while (n > 0) {
+   digit <- n %% 10
+   reverse <- reverse * 10 + digit
+   n <- n %/% 10
+ }
>
> print(reverse)
[1] 4321
```

5. Generate random numbers until a value greater than 90 appears

```
attempts <- 0
repeat {
  x <- sample(1:100, 1)
  attempts <- attempts + 1
  print(x)

  if (x > 90) {
    break
  }
}
print(paste("Attempts:", attempts))
```

```
[1] 2
[1] 22
[1] 62
[1] 52
[1] 7
[1] 11
[1] 82
[1] 78
[1] 11
[1] 20
[1] 68
[1] 81
[1] 6
[1] 66
[1] 100
>
> print(paste("Attempts:", attempts))
[1] "Attempts: 15"
```

6. Double positive values in a vector

```
double_positive <- function(x) {
  for (i in 1:length(x)) {
    if (x[i] > 0) {
      x[i] <- x[i] * 2
```

```
  }
 }
 return(x)
}
print(double_positive(c(2, -3, 0, 5)))
```

```
> double_positive <- function(x) {
+   for (i in 1:length(x)) {
+     if (x[i] > 0) {
+       x[i] <- x[i] * 2
+     }
+   }
+   return(x)
+ }
>
> print(double_positive(c(2, -3, 0, 5)))
[1]  4 -3  0 10
>
```

7. Classify students based on marks

```
result <- function(mark) {
 if (mark >= 75) {
  return("Distinction")
 } else if (mark >= 60) {
  return("First Class")
 } else if (mark >= 40) {
  return("Pass")
 } else {
  return("Fail")
 }
}
print(result(85))
print(result(65))
print(result(45))
print(result(30))
```

```
+ }
>
> print(result(85))
[1] "Distinction"
> print(result(65))
[1] "First Class"
> print(result(45))
[1] "Pass"
> print(result(30))
[1] "Fail"
>
```

8. Simple interest using default arguments

```
simple_interest <- function(p, r = 5, t = 2) {
  si <- (p * r * t) / 100
  return(si)
}
print(simple_interest(10000))
print(simple_interest(10000, 8, 3))
```

```
> simple_interest <- function(p, r = 5, t = 2) {
+   si <- (p * r * t) / 100
+   return(si)
+ }
>
> print(simple_interest(10000))
[1] 1000
> print(simple_interest(10000, 8, 3))
[1] 2400
```

9. Skip multiples of 5 using next

```
for (i in 1:50) {
  if (i %% 5 == 0) {
    next
  }
  cat(i, " ")
```

}

```
> for (i in 1:50) {
+   if (i %% 5 == 0) {
+     next
+   }
+   cat(i, " ")
+ }
1  2  3  4  6  7  8  9  11  12  13  14  16  17  18  19  21  22  23  24  26  27  28  29
31  32  33  34  36  37  38  39  41  42  43  44  46  47  48  49
>
```

10.Find the first number divisible by both 7 and 11

```
start <- 1
end <- 100
found <- FALSE
for (i in start:end) {
  if (i %% 7 == 0 && i %% 11 == 0) {
    print(i)
    found <- TRUE
    break
  }
}
if (!found) {
  print("No number found")
}
```

```
> found <- FALSE
>
> for (i in start:end) {
+   if (i %% 7 == 0 && i %% 11 == 0) {
+     print(i)
+     found <- TRUE
+     break
+   }
+ }
[1] 77
>
> if (!found) {
+   print("No number found")
+ }
```

11.Sales statistics using a named list

```
sales_summary <- function(sales) {
  return(list(
    Total = sum(sales),
    Average = mean(sales),
    Maximum = max(sales),
    Minimum = min(sales)
  ))
}
print(sales_summary(c(100, 200, 300, 400)))
```

```
+     Maximum = max(sales),
+     Minimum = min(sales)
+   ))
+ }
>
> print(sales_summary(c(100, 200, 300, 400)))
$Total
[1] 1000

$Average
[1] 250

$Maximum
[1] 400

$Minimum
[1] 100
```

12.**Check whether a number is prime**

```
is_prime <- function(n) {
  if (n < 2) {
    return(FALSE)
  }
  for (i in 2:floor(sqrt(n))) {
    if (n %% i == 0) {
```

```
    return(FALSE)
   }
  }

  return(TRUE)
}

print(is_prime(7))
print(is_prime(10))
print(is_prime(13))
```

```
+      }
+    }
+
+    return(TRUE)
+ }
>
> print(is_prime(7))
[1] TRUE
> print(is_prime(10))
[1] FALSE
> print(is_prime(13))
[1] TRUE
```

13.**Factorial using recursion**

```
factorial_rec <- function(n) {
  if (n == 0) {
    return(1)
  }
  return(n * factorial_rec(n - 1))
}
print(factorial_rec(5))
print(factorial_rec(0))
```

```
+   }
+
+   return(n * factorial_rec(n
+ }
>
> print(factorial_rec(5))
[1] 120
> print(factorial_rec(0))
[1] 1
>
```

14.**Sum of the first n natural numbers using recursion**

```
sum_n <- function(n) {
  if (n == 0) {
    return(0)  # Base case stops recursion
  }
  return(n + sum_n(n - 1))
}
print(sum_n(5))
print(sum_n(10))
```

```
+
+   return(n + sum_n(n -
+ }
>
> print(sum_n(5))
[1] 15
> print(sum_n(10))
[1] 55
>
```

15.Find the nth Fibonacci number using recursion

```
fibonacci <- function(n) {
  if (n < 0 || n != floor(n)) {
```

```
  return("Invalid input")
 }
 if (n == 0) {
  return(0)
 }
 if (n == 1) {
  return(1)
 }
 return(fibonacci(n - 1) + fibonacci(n - 2))
}
print(fibonacci(0))
print(fibonacci(5))
print(fibonacci(7))
```

```
+   if (n == 0) {
+     return(0)
+   }
+
+   if (n == 1) {
+     return(1)
+   }
+
+   return(fibonacci(n - 1) + fibonacci(n - 2))
+ }
>
> print(fibonacci(0))
[1] 0
> print(fibonacci(5))
[1] 5
> print(fibonacci(7))
[1] 13
>
```

16.Find GCD using the Euclidean algorithm

```
a <- 24
b <- 18
```

```
while (b != 0) {
  temp <- b
  b <- a %% b
  a <- temp
}
print(paste("GCD =", a))
```

```
> a <- 24
> b <- 18
>
> while (b != 0) {
+   temp <- b
+   b <- a %% b
+   a <- temp
+ }
>
> print(paste("GCD =", a))
[1] "GCD = 6"
>
```

17.Electricity bill calculation

```
bill <- function(units) {
  if (units <= 100) {
    amount <- units * 2
  } else if (units <= 200) {
    amount <- 100 * 2 + (units - 100) * 3
  } else {
    amount <- 100 * 2 + 100 * 3 +
              (units - 200) * 5
  }
  return(amount)
}
```

print(bill(250))

```
> bill <- function(units) {
+   if (units <= 100) {
+     amount <- units * 2
+   } else if (units <= 200) {
+     amount <- 100 * 2 + (units - 100) * 3
+   } else {
+     amount <- 100 * 2 + 100 * 3 +
+       (units - 200) * 5
+   }
+
+   return(amount)
+ }
>
> print(bill(250))
[1] 750
```

18. **Count passed, failed, and high-scoring students**

```
marks <- c(85, 35, 60, 78, 25, 45)
pass <- 0
fail <- 0
high <- 0

for (m in marks) {
  if (m >= 40) {
    pass <- pass + 1
  } else {
    fail <- fail + 1
  }

  if (m >= 75) {
    high <- high + 1
  }
}

print(paste("Passed:", pass))
print(paste("Failed:", fail))
print(paste("Scored 75 or above:", high))
```

```
+     pass <- pass + 1
+   } else {
+     fail <- fail + 1
+   }
+
+   if (m >= 75) {
+     high <- high + 1
+   }
+ }
>
> print(paste("Passed:", pass))
[1] "Passed: 4"
> print(paste("Failed:", fail))
[1] "Failed: 2"
> print(paste("Scored 75 or above:", high))
[1] "Scored 75 or above: 2"
```

19.Filter values using a threshold

```
filter_values <- function(x, threshold = 50) {
  return(x[x >= threshold])
}

print(filter_values(c(20, 50, 60, 80)))
print(filter_values(c(20, 50, 60, 80), 60))
```

```
> filter_values <- function(x, threshold = 50)
+   return(x[x >= threshold])
+ }
>
> print(filter_values(c(20, 50, 60, 80)))
[1] 50 60 80
> print(filter_values(c(20, 50, 60, 80), 60))
[1] 60 80
```

20.Return NULL if a vector contains missing values

```
square_values <- function(x) {
  if (any(is.na(x))) {
    return(NULL)
  } else {
    return(x * x)
  }
}
```

print(square_values(c(2, 3, 4)))

print(square_values(c(2, NA, 4)))

```
> square_values <- function(x) {
+   if (any(is.na(x))) {
+     return(NULL)
+   } else {
+     return(x * x)
+   }
+ }
>
> print(square_values(c(2, 3, 4)))
[1]  4  9 16
> print(square_values(c(2, NA, 4)))
NULL
```

21.Parcel delivery charge using nested if-else

```
delivery_charge <- function(weight, distance) {
  if (weight <= 2) {
    if (distance <= 10) {
      charge <- 50
    } else {
      charge <- 50 + ceiling((distance - 10) / 10) * 10
    }
  } else {
    if (distance <= 10) {
      charge <- 100
    } else {
      charge <- 100 + ceiling((distance - 10) / 10) * 20
    }
  }
  return(charge)
}
```

print(delivery_charge(1.5, 25))

print(delivery_charge(5, 25))

```
+     } else {
+       charge <- 50 + ceiling((distance - 10) / 10) * 10
+     }
+   } else {
+     if (distance <= 10) {
+       charge <- 100
+     } else {
+       charge <- 100 + ceiling((distance - 10) / 10) * 20
+     }
+   }
+
+   return(charge)
+ }
>
> print(delivery_charge(1.5, 25))
[1] 70
> print(delivery_charge(5, 25))
[1] 140
```

22.**Multiplication table from 1 to 12**

n <- as.integer(readline("Enter a number: "))

for (i in 1:12) {

  cat(n, "x", i, "=", n * i, "\n")

}

```
> n <- as.integer(readline("Enter a number: "))
Enter a number:
> for (i in 1:12) {
+   cat(n, "x", i, "=", n * i, "\n")
+ }
NA x 1 = NA
NA x 2 = NA
NA x 3 = NA
NA x 4 = NA
NA x 5 = NA
NA x 6 = NA
NA x 7 = NA
NA x 8 = NA
NA x 9 = NA
NA x 10 = NA
NA x 11 = NA
NA x 12 = NA
```

23.Count website visits above the average

```
visit_summary <- function(visits) {
  avg <- mean(visits)
  count <- 0
  for (v in visits) {
    if (v > avg) {
      count <- count + 1
    }
  }
  return(list(
    Average = avg,
    Days_Above_Average = count
  ))
}
print(visit_summary(c(100, 200, 300, 400, 500, 600, 700)))
```

```
+     if (v > avg) {
+       count <- count + 1
+     }
+   }
+
+   return(list(
+     Average = avg,
+     Days_Above_Average = count
+   ))
+ }
>
> print(visit_summary(c(100, 200, 300, 400, 500, 600, 700)))
$Average
[1] 400

$Days_Above_Average
[1] 3
```

24.Calculate power without using ^ or **

```
x <- 2
n <- 5
if (is.na(n) || n < 0 || n != floor(n)) {
  print("Invalid exponent")
} else {
  result <- 1

  for (i in 1:n) {
    result <- result * x
  }
  print(result)
}
```

```
> x <- 2
> n <- 5
>
> if (is.na(n) || n < 0 || n != floor(n)) {
+   print("Invalid exponent")
+ } else {
+   result <- 1
+
+   for (i in 1:n) {
+     result <- result * x
+   }
+
+   print(result)
+ }
[1] 32
```

25.Find the maximum value using recursion

```
find_max <- function(x) {
  if (length(x) == 1) {
    return(x[1])
```

```
  }
  rest_max <- find_max(x[-1])
  if (x[1] > rest_max) {
    return(x[1])
  } else {
    return(rest_max)
  }
}
print(find_max(c(10, 25, 8, 40, 15)))
print(find_max(c(7)))
```

```
> find_max <- function(x) {
+   if (length(x) == 1) {
+     return(x[1])
+   }
+
+   rest_max <- find_max(x[-1])
+
+   if (x[1] > rest_max) {
+     return(x[1])
+   } else {
+     return(rest_max)
+   }
+ }
>
> print(find_max(c(10, 25, 8, 40, 15)))
[1] 40
> print(find_max(c(7)))
[1] 7
```